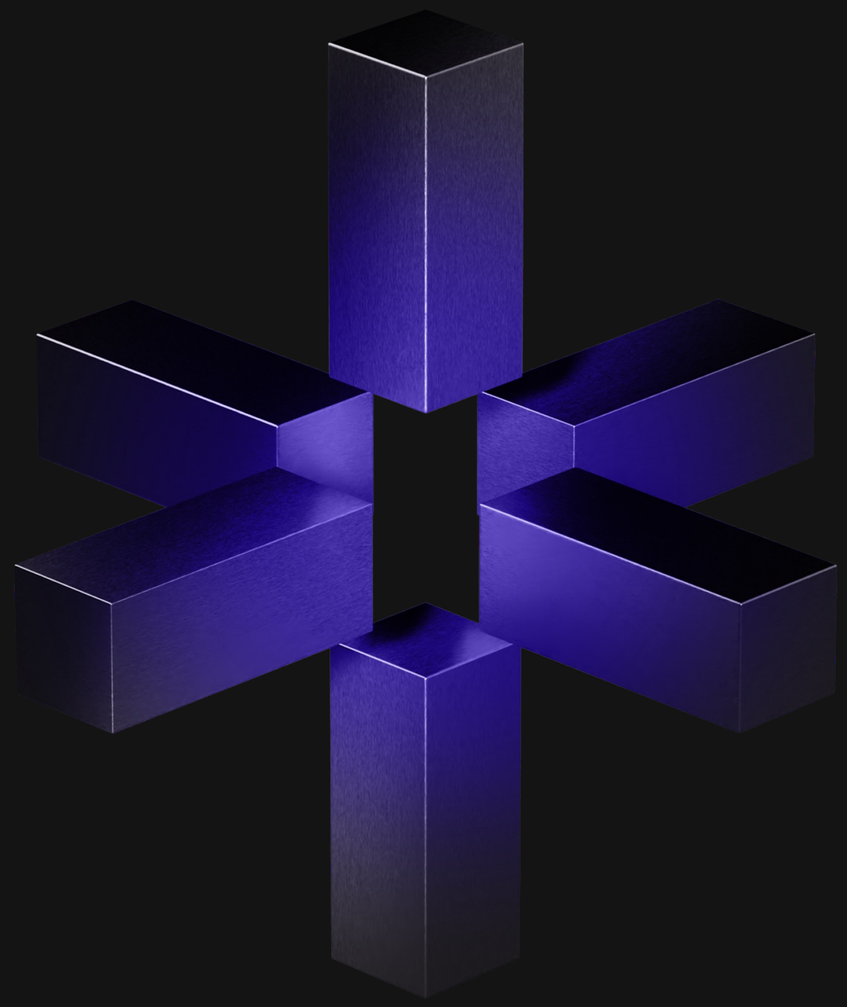
ЦЕНТРАЛЬНЫЙ УНИВЕРСИТЕТ

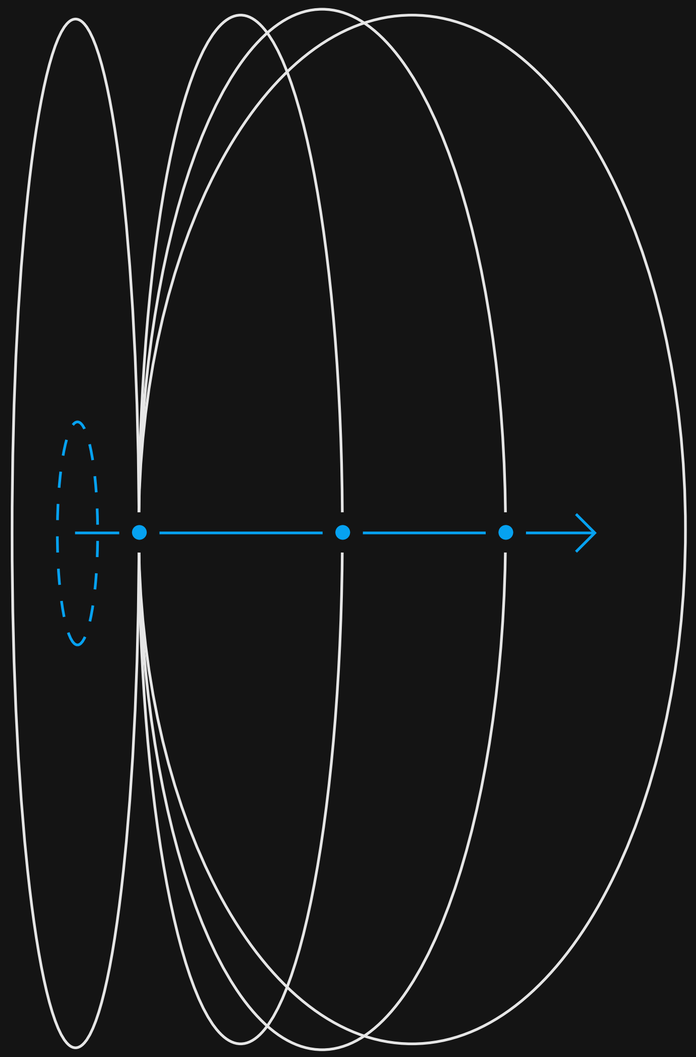


Метрические алгоритмы

ML
Машинное обучение

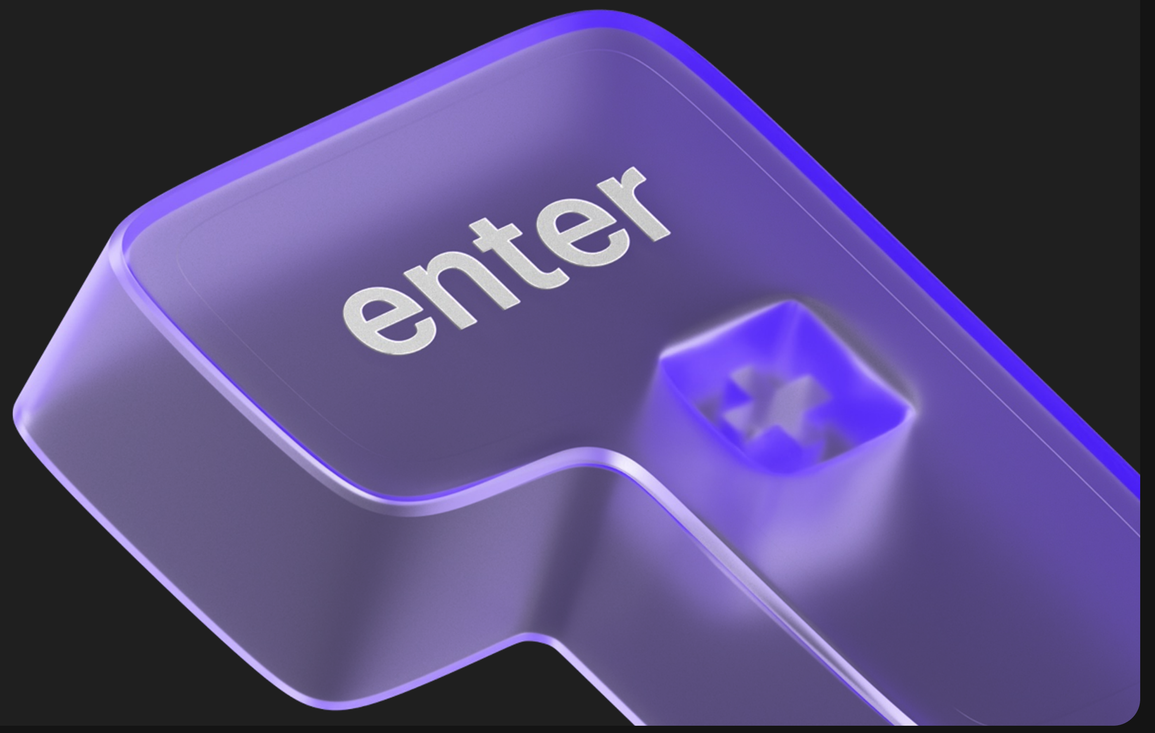
03

План лекции



- Метрические алгоритмы
- Расстояние
- Ближайший центроид
- Алгоритм ближайших соседей (kNN)

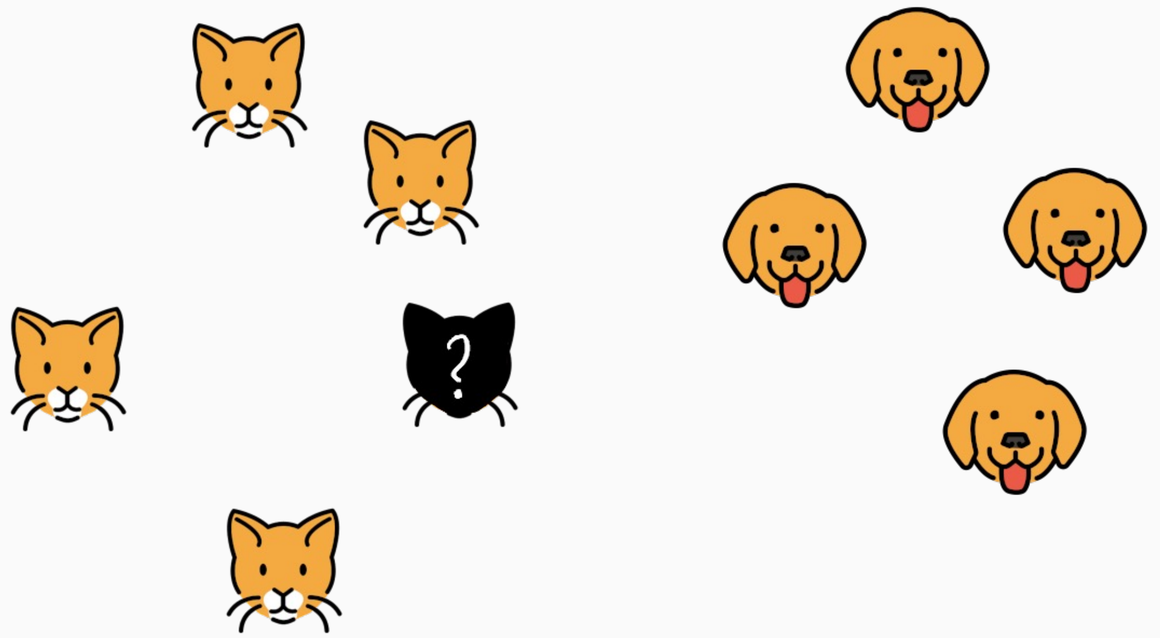
Метрические алгоритмы



Метрические алгоритмы

Чтобы определить свойства нового объекта, мы сравниваем его с теми, что уже видели.

Метрические алгоритмы находят в выборке наиболее близких соседей и переносят их свойства на новый объект.



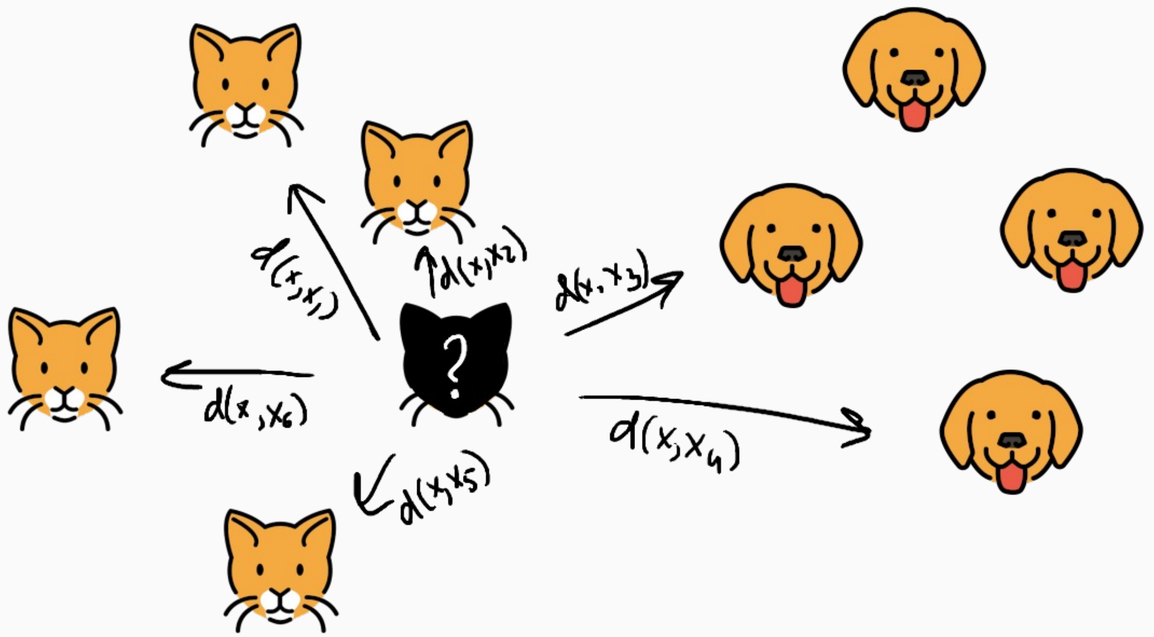
Источник

Метрические алгоритмы

Метрические алгоритмы сравнивают расстояние между объектами в пространстве признаков.

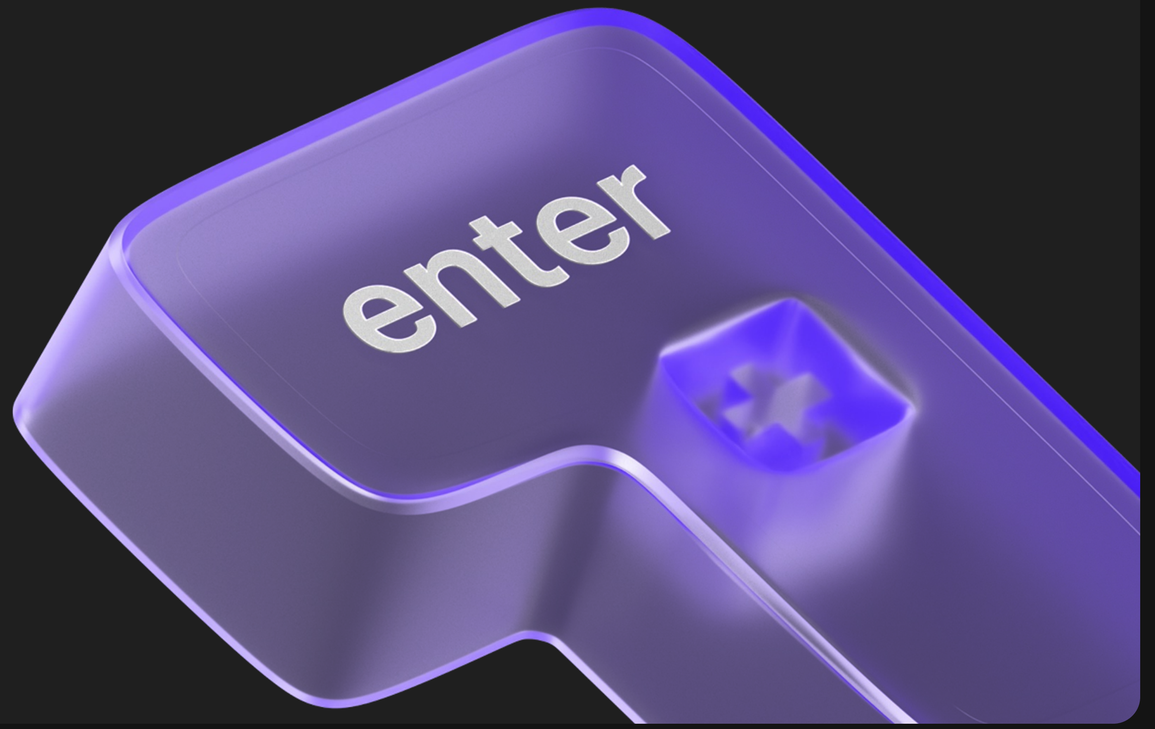
Основные подходы

- → Сравнение с ближайшим центроидом (nearest centroid)
- → Анализ k ближайших соседей (kNN)



Источник

Расстояние



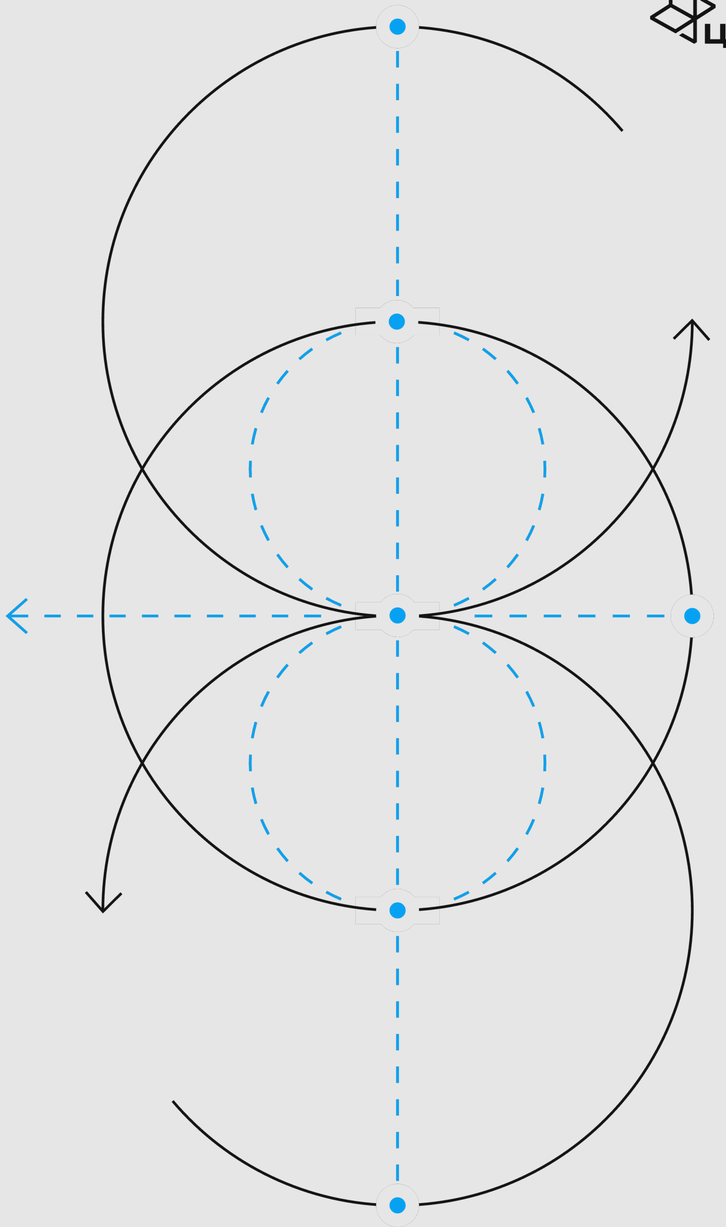
Расстояние

Расстояние (или метрика) — это функция $d(x, y) : X \times X \to R.$

Она измеряет близость между объектами в пространстве признаков.

Функция расстояния $d(x, y)$ отвечает следующим условиям:

- **Неотрицательность**
  $d(x, y) \geq 0$
- **Симметричность**
  $d(x, y) = d(y, x)$
- **Тождество неразличных**
  $d(x, y) = 0 \Leftrightarrow x = y$
- **Неравенство треугольника**
  $d(x, y) \leq d(x, z) + d(z, y)$



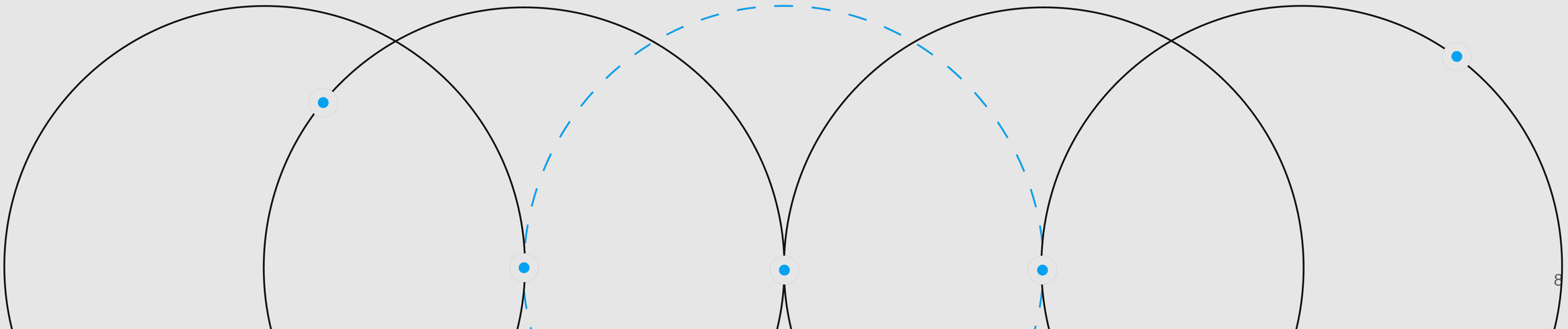
Расстояние

Чаще в работе используют:

- Минковского
- Манхэттенское
- Евклидово
- Косинусное
- Хэмминга

Для специфичных задач используют:

- Махаланобиса
- Canberra distance
- Жаккара
- DTW
- Левенштейна



Расстояние

Минковского (Lp)

Обобщённая метрика расстояния.

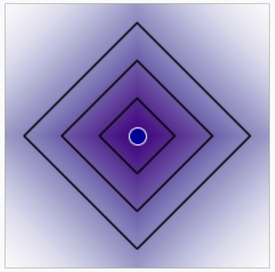
$$d_p(x,\ y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p\right)^{\frac{1}{p}}$$

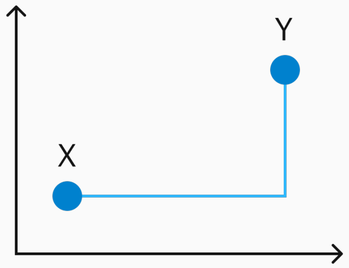
В ML расстояние Минковского достаточно чувствительно к масштабу признаков.

Манхэттенское (L1)

Соответствует движению по городу с прямоугольной сеткой улиц.

$$d_1(x,\ y) = \sum_{i=1}^{n} |x_i - y_i|$$

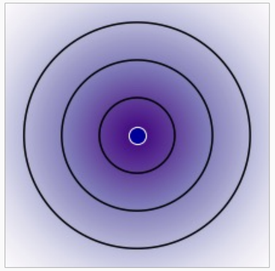


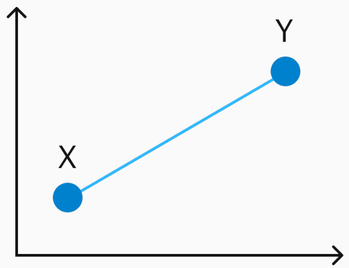


Евклидово (L2)

Длина кратчайшего пути между двумя точками в пространстве.

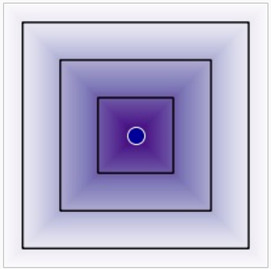
$$d_2(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$





Чебышёва (L∞) — предельный случай Максимальное расстояние по любой из координат.

$$d_\infty(x, y) = \max_{1 \le i \le n} |x_i - y_i|$$



Источник

Расстояние

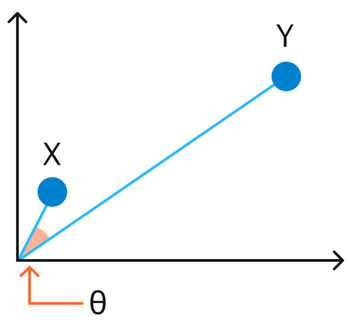
Косинусное

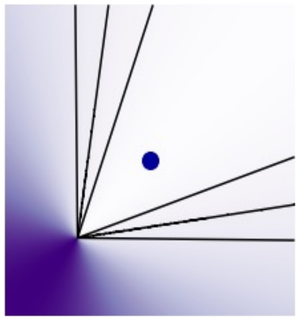
Угловое сходство между двумя векторами.

$$d_{cos}(x, y) = 1 - \cos(\theta)$$

$\cos(\theta)$ — **косинусное сходство.**

$$\cos(\theta) = \frac{xy}{||x||||y||} = \frac{\sum_{i=1}^{n} x_i y_i}{\sqrt{\sum_{i=1}^{n}(x_i^2)}\sqrt{\sum_{i=1}^{n}(y_i^2)}}$$





Хэмминга

Сравнивает два вектора (слова, списка) одинаковой длины и считает количество различающихся соответственных символов.

$$d_H(x, y) = \sum_{i=1}^{n} I(x_i \neq y_i)$$

Источник

Как выбрать подходящую метрику

Метрика подбирается эмпирически, но вдумчиво.
Её выбор зависит от задачи и типа данных.

Числовые непрерывные признаки (векторы в R^n)

Обычно используется: евклидово

Если есть выбросы или высокая размерность: манхэттенское

Очень важно масштабировать признаки.

Важны пропорции, а не абсолютное значение

Косинусное

Например:

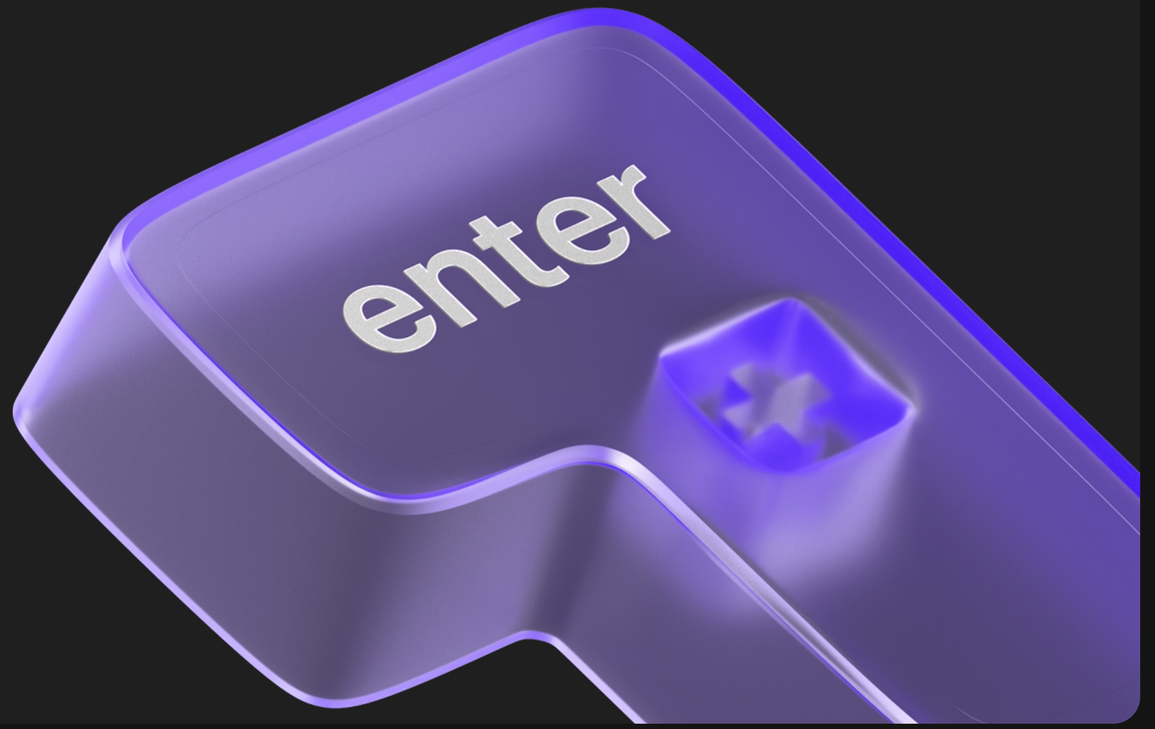
- текстовые данные
- векторы предпочтений пользователей в RecSys
- изображения в компьютерном зрении

Категориальные признаки

Хэмминга

Другие метрики используются для более специфичных задач.

Метод ближайшего центроида



В чём идея

→ **Выборка**

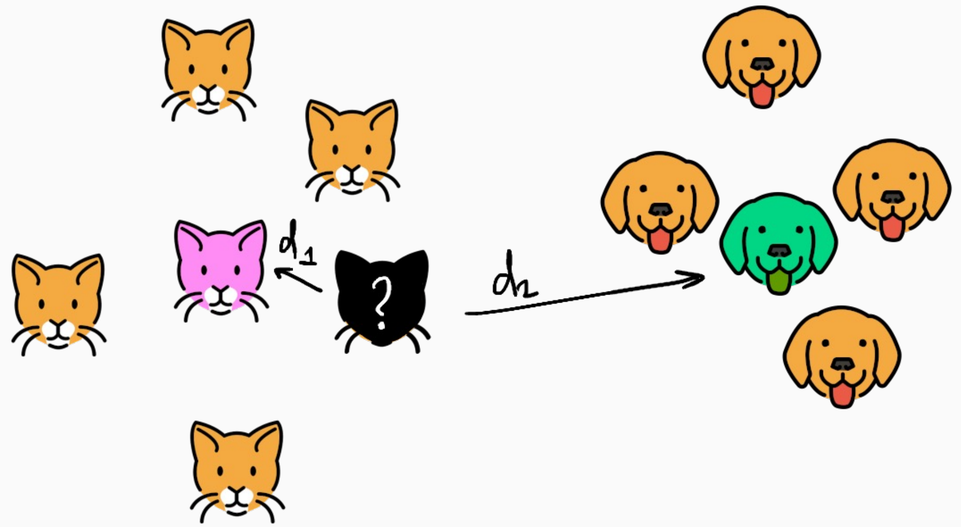
$$\{(x_1, y_1), \ldots, (x_n, y_n)\} \quad y_i \in K = \{1, \ldots, k\}$$

→ **Центроид**

$$\mu_k = \frac{1}{|\{i | y_i = k\}|} \sum_{i \in I_k} x_i$$

→ **Предсказание**

$$\hat{y} = \arg\min_{l \in K} d(\vec{x}, \mu_l)$$



Объект относится к тому классу, чей центроид находится ближе всего к нему в признаковом пространстве.

Источник

Особенности метода

Хранятся только центроиды

Их можно адаптивно менять.

Быстрое предсказание

Сравниваем новый объект только с k центроидами.

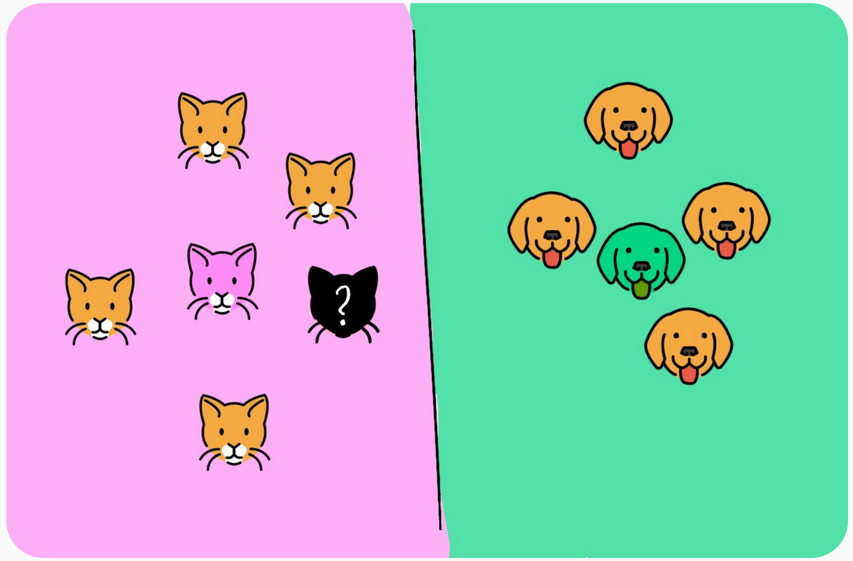
Небольшой размер модели

k классов × описание центроида.

Очень простой алгоритм

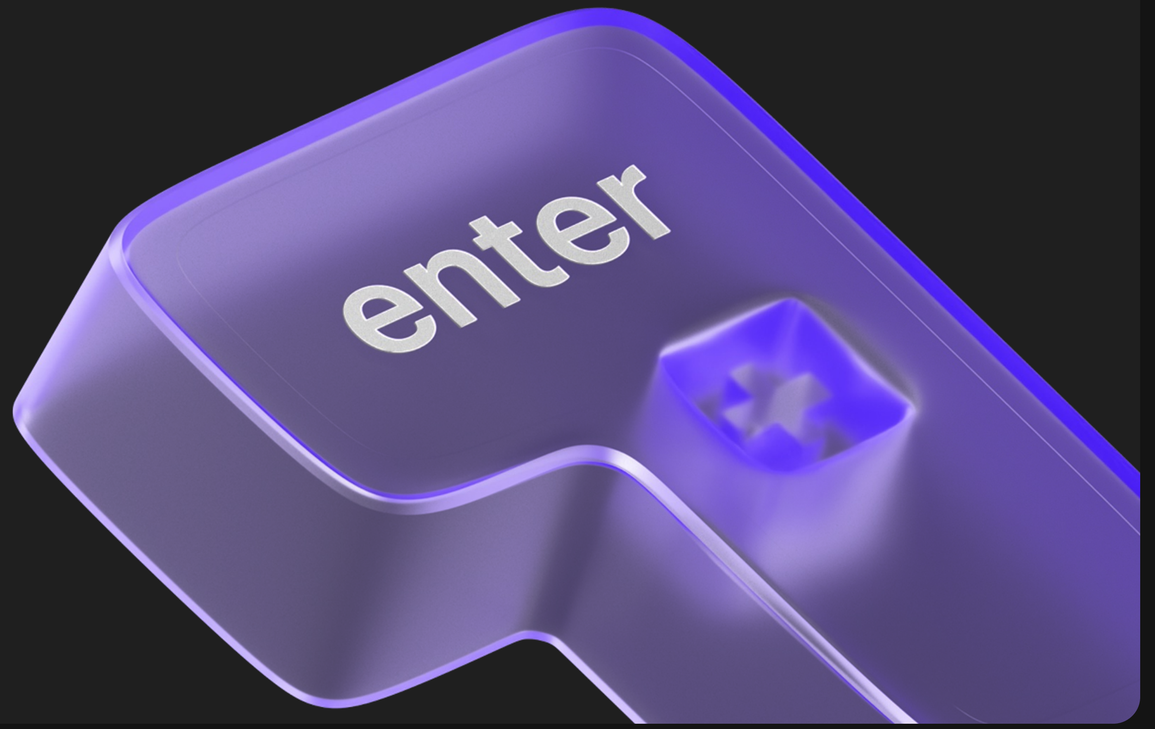
Подходит только для линейно разделимых классов.

Плохо работает с данными со сложной структурой и различным распределением.



Источник

Алгоритм ближайших соседей (kNN)

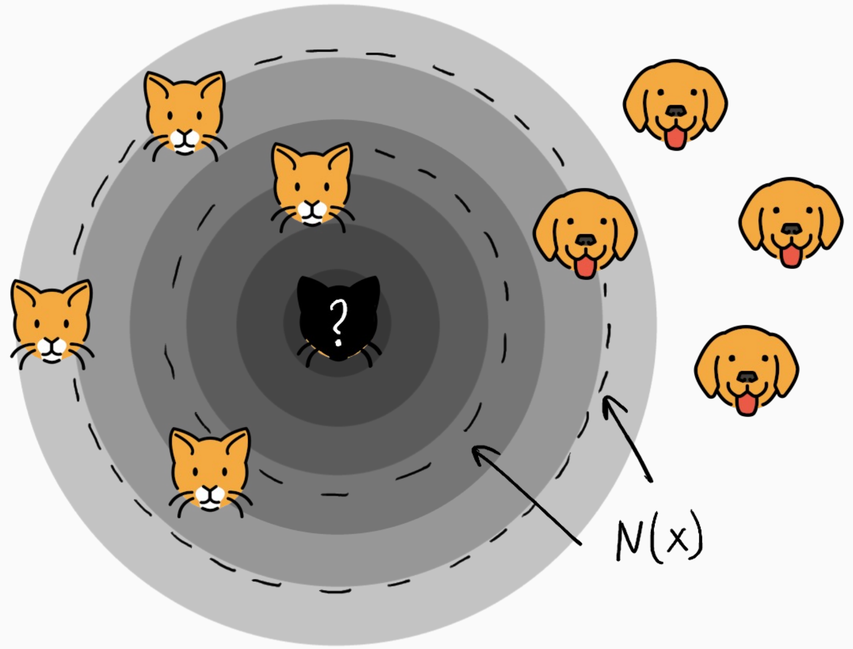


kNN (англ. k-nearest neighbors)

Подход, который основан на близости объектов.

→ Используется для задач классификации и регрессии.

→ Находит k ближайших соседей нового объекта в обучающей выборке и делает предсказание на основе их свойств.



N (x) — окрестность вокруг точки

Источник

Окрестность в kNN

Окрестность можно задать фиксированным радиусом, но подбирать его неудобно.

Поэтому:

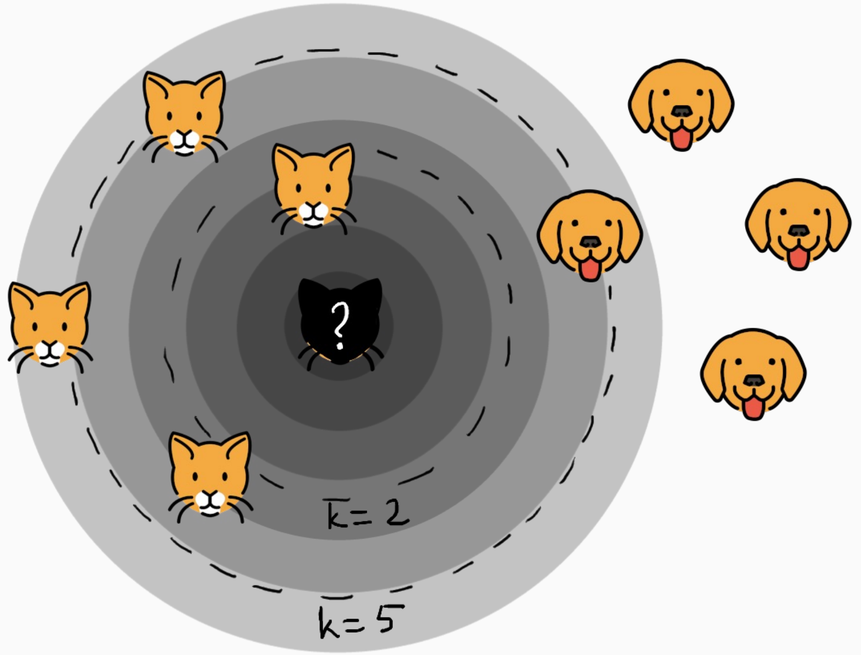
пусть x — новый объект;

$d(x, x_n)$ — расстояния до других объектов выборки;

при этом $d(x, x_1) < \ldots < d(x, x_n)$.

Тогда окрестность:

$N(x) = \{x_1, \ldots, x_k\}$ — область, включающая k ближайших соседей.



Источник

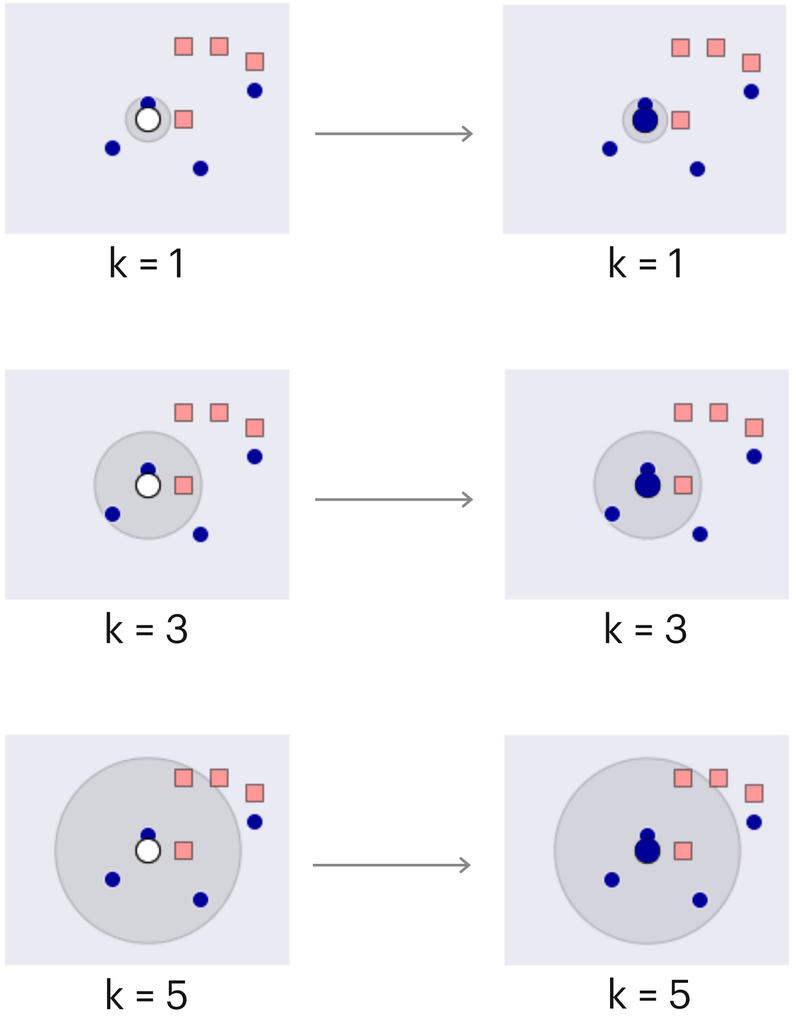
kNN для задачи классификации

Majority vote: объекту приписывается класс, который чаще всего встречается среди соседей.

Гиперпараметр k отвечает за устойчивость к выбросам.

$$\hat{y} = mode\left(y_j \middle| (x_j,\ y_j) \in N_k(\vec{x})\right) = \text{argmax}_{k \in K} \sum_{(x_j,\ y_j) \in N_k(\vec{x})} I(\ y_j = k)$$

В бинарной классификации, как правило, k нечётное.



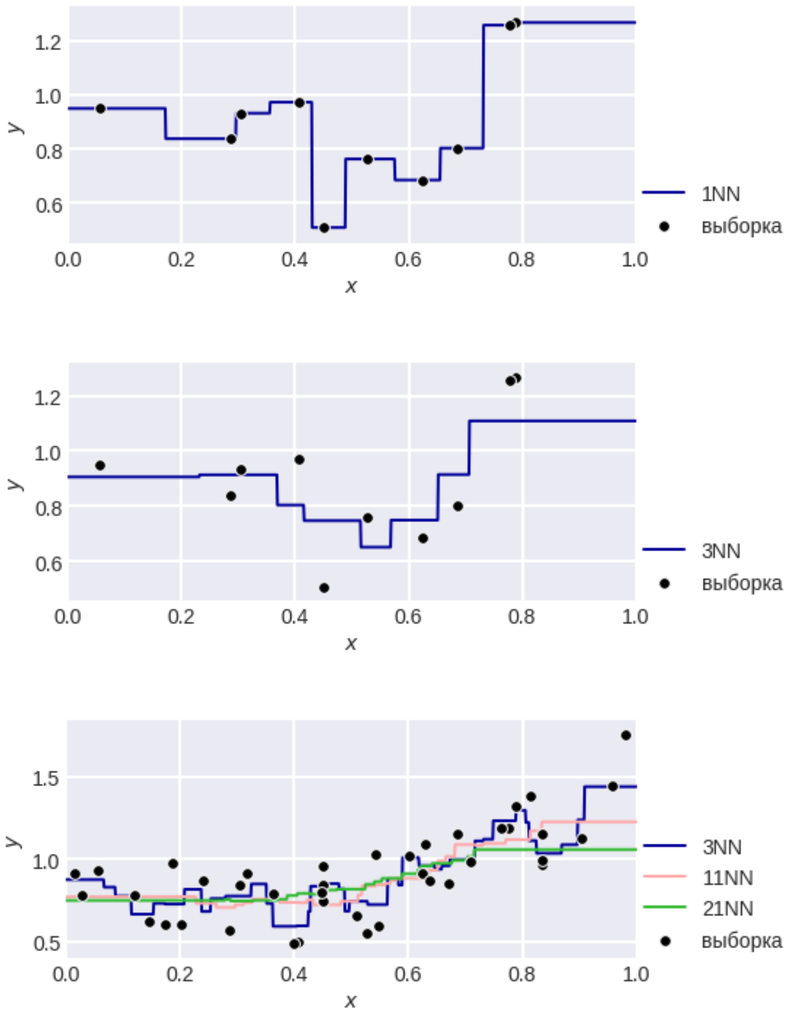
Источник.

kNN для задачи регрессии

Предсказание для объекта — среднее значение k соседей.

Гиперпараметр k отвечает за степень сглаживания.

$$\hat{y} = mean\Big(y_j \Big| (x_j,\ y_j) \in N_k(\vec{x})\Big) = \frac{1}{k} \sum_{(x_j,\ y_j) \in N_k(\vec{x})} y_j$$



Источник.

Гиперпараметр k

Отвечает за локальность алгоритма.

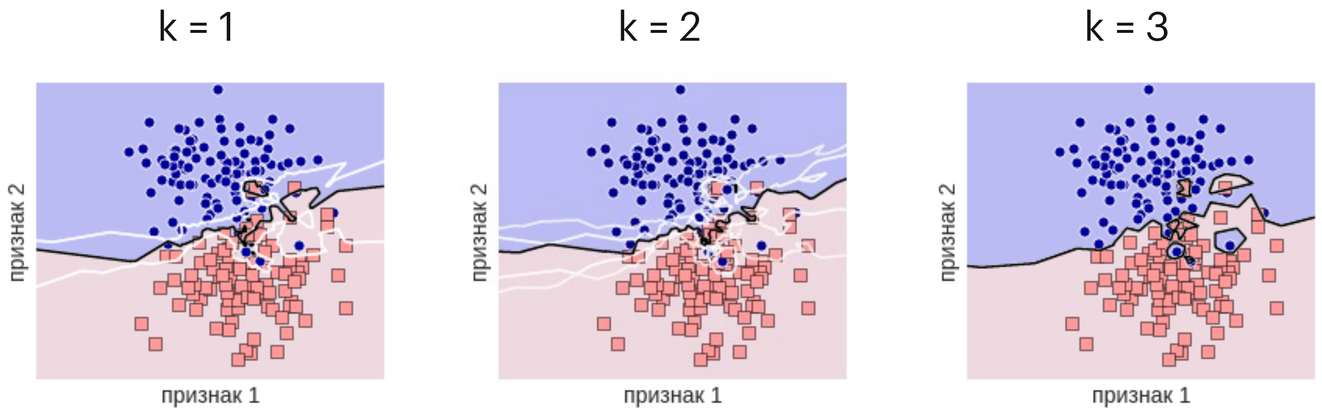
Оптимальное значение аналитически определить нельзя. Нужно подбирать с помощью кросс-валидации.

Маленький k

- Чувствительность к шуму.
- Сложная граница решений, которая подстраивается под каждый объект.
- Риск переобучения.

Большой k

- Устойчивость к шуму.
- Граница решений гладкая.
- Потеря локальных паттернов.
- Риск недообучения.



Источник

Взвешенный kNN

Главная идея: чем ближе соседи,
тем больше их вес.

В задаче классификации:

$$\hat{y} = \text{argmax}_{k \in K} \sum_{(x_j,\, y_j) \in N_k(\vec{x})} w_j \times I(\, y_j = k)$$

В задаче регрессии:

$$\hat{y} = \frac{\sum_{(x_j,\, y_j) \in N_k(\vec{x})} w_j y_j}{\sum_{(x_j,\, y_j) \in N_k(\vec{x})} w_j}$$



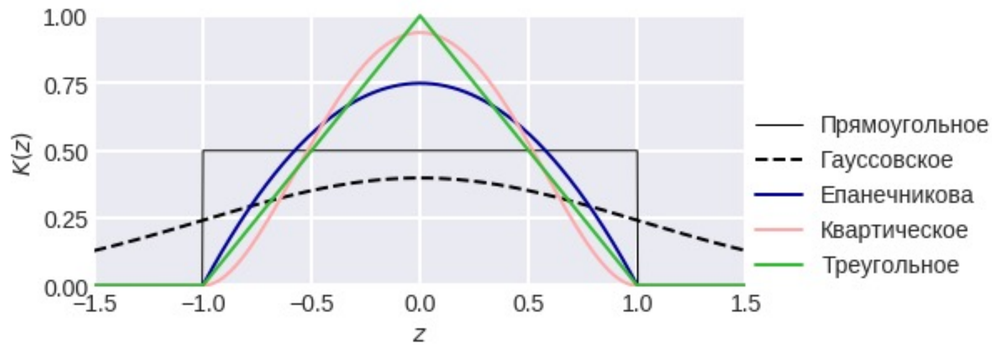
Источник

Схемы взвешивания

Ядерная функция (англ. kernel function)
преобразует расстояние в вес.
Чем ближе объект, тем больше его вес.

Веса: $w_j = K(d(\vec{x}\ ,\ x_j))$

Оптимальное значение аналитически определить нельзя. Нужно подбирать с помощью кросс-валидации.



→ **Гауссовское ядро (Gaussian kernel)** $\frac{1}{\sqrt{2\pi}}\exp(-\frac{1}{2}d^2)$

→ **Треугольное (triangular, linear)** $(1-|d|)\times I(|d|\leq 1)$

→ **Прямоугольное (rectangular, top-hat)** $\frac{1}{2}I(|d|\leq 1)$

→ **Епанечникова** $\frac{3}{4}(1-d^2)\cdot I(|d|\leq 1)$

→ **Косинусное** $\frac{\pi}{4}\cos(\frac{\pi}{2}d)\times I(|d|\leq 1)$

→ **Квартическое (quartic)** $\frac{15}{16}(1-d^2)^2\times I(|d|\leq 1)$

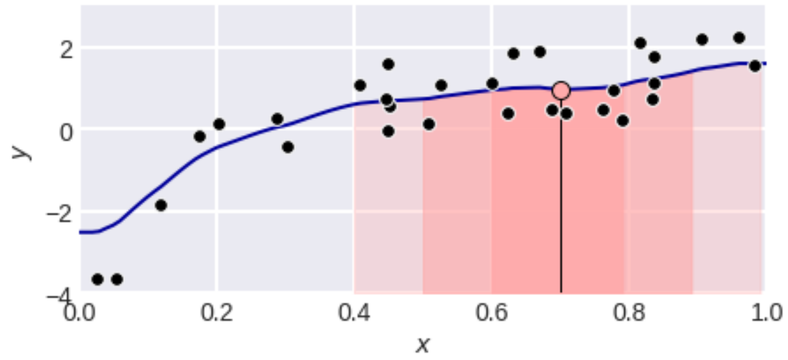
Источник

Регрессия Надарая — Ватсона

Метод, который аппроксимирует данные с помощью ядерного сглаживания.
Его можно рассматривать как обобщение взвешенного kNN.

Используем все точки выборки, а не только k соседей:

$$\hat{y}(x) = \frac{\sum_{i=1}^{n} K\left(\frac{d(\vec{x}, x_i)}{h}\right) y_i}{\sum_{i=1}^{n} K\left(\frac{d(\vec{x}, x_i)}{h}\right)}$$



Не экстраполирует.

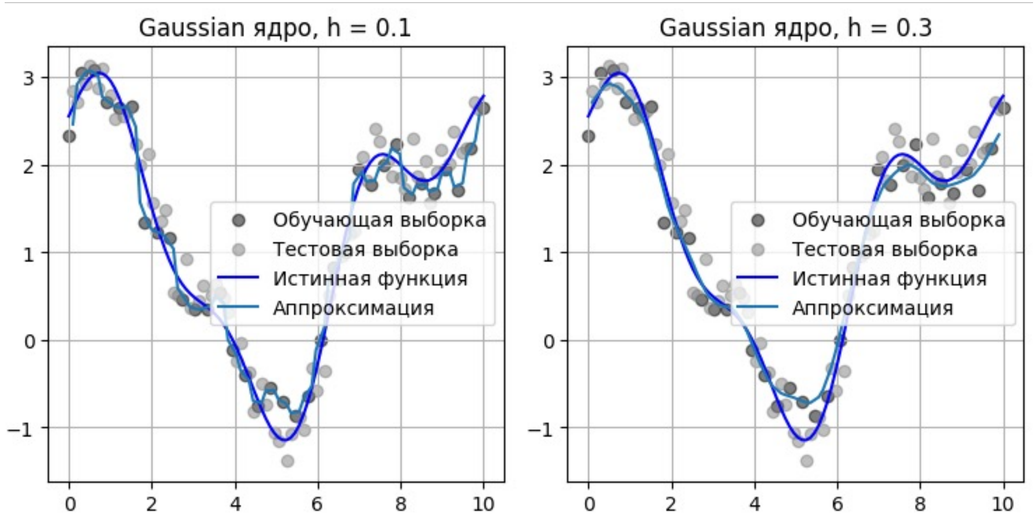
Источник

Гиперпараметр h

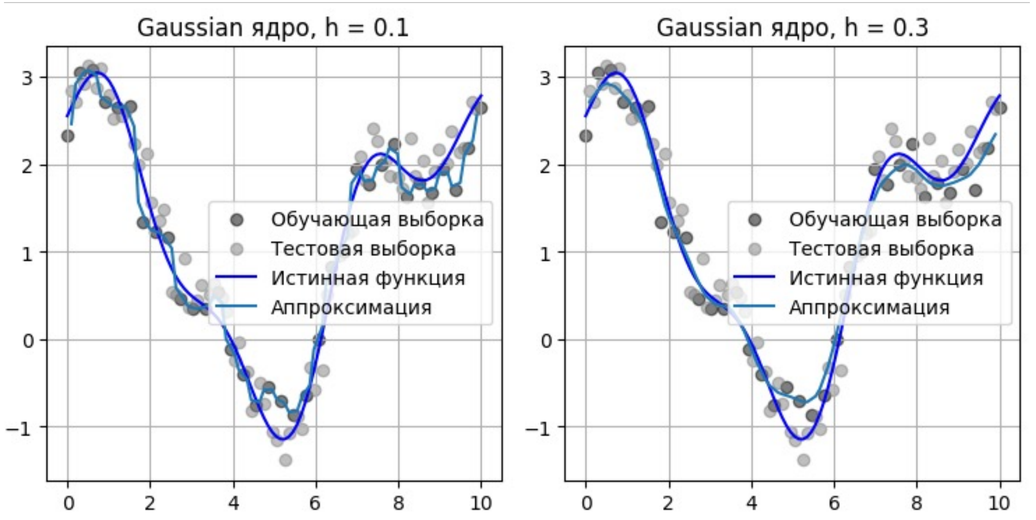
Ширина окна (англ. bandwidth) — гиперпараметр h, который отвечает за радиус влияния ядра.

h — это аналог k в kNN.

Маленький h — риск переобучения.



Большой h — риск недообучения.



Источник

Особенности kNN

- ⊕ Прост в понимании и реализации
- ⊕ Непараметрический
  Не делаем предположений о форме зависимости.
- ⊕ Улавливает локальные паттерны
  Может хорошо моделировать сложные границы решений.
- ⊕ Легко адаптируется к новым данным

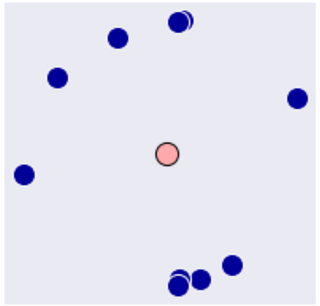
- ⊖ Вычислительно дорогой
- ⊖ Требователен к памяти
  Большие датасеты могут не поместиться в RAM.
- ⊖ Чувствителен к выбору метрики и масштабированию
- ⊖ Не существует универсального k
  Каждый раз подбираем заново.

Проклятие размерности (curse of dimensionality)

В пространстве с большим числом признаков:

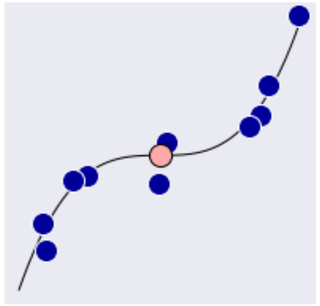
→ Все точки становятся примерно равноудалены друг от друга.

→ Данные сильно разрежены.



Теоретически эффективность kNN падает.

Данные имеют внутреннюю структуру: они лежат
на низкоразмерном многообразии, с которым и работает алгоритм.



Источник

Оптимизации kNN

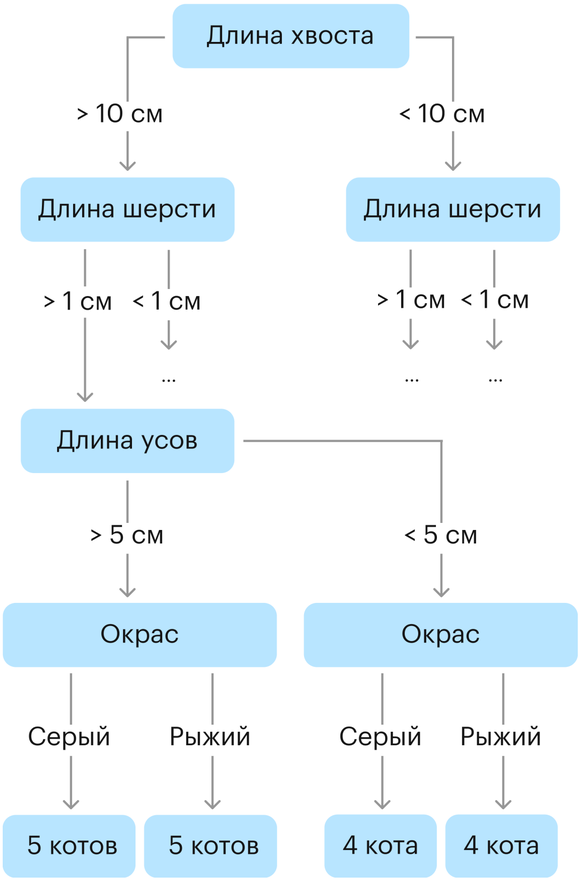
В промышленной среде используются более эффективные реализации: методы приближенного поиска ближайших соседей, или **ANN (англ. approximate nearest neighbors).**

Примеры методов:

- KD-деревья
- Locality-sensitive hashing (LSH)
- Hierarchical navigable small worlds (HNSW) — SoTA

Пример реализации:

- Annoy (Spotify)



Не теряя в качестве, можно сильно ускорить процесс предсказания. Как именно — разберём на семинарах.

Итоги

Ключевое понятие: метрические алгоритмы — методы машинного обучения, которые делают предсказание для объекта на основе сравнения с соседями.

Расстояние — главный инструмент метрических алгоритмов.

Предсказание делается на основе свойств соседей.

Метрические алгоритмы — непараметрические.

Идея соседства обобщается на разные типы задач.

Классификация: majority vote.

Регрессия: среднее значение.

Итоги

Выбор параметров добавляет гибкости.

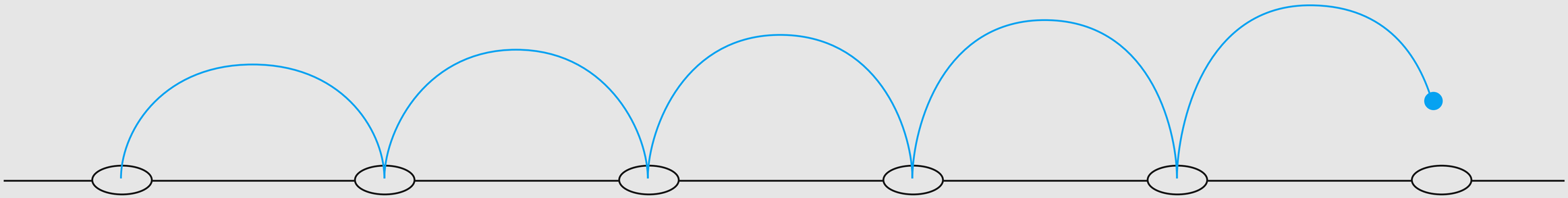
Выбор метрики позволяет адаптировать алгоритм под тип данных.

Выбор k: баланс между обобщением и локальностью.

Минусы: высокая вычислительная сложность.

...Но!

Существуют быстрые и эффективные реализации для поиска соседей.





ЦЕНТРАЛЬНЫЙ УНИВЕРСИТЕТ



Вопросы